

Subqueries

Objectives

- After completing this lesson, you should be able to do the following:
  - Describe the types of problems that subqueries can solve
  - Define subqueries
  - List the types of subqueries
  - Write single-row and multiple-row subqueries

Using a Subquery to Solve a Problem

- "Who has a salary greater than Jones'?"

Main Query



"Which employees have a salary greater than Jones' salary?"

Subquery



"What is Jones' salary?"

Subqueries

```
SELECT      select_list
FROM        table
WHERE       expr operator
                    (SELECT   select_list
                  FROM                table);
```

- The subquery (inner query) executes once before the main query.
- The result of the subquery is used by the main query (outer query).

Subqueries

A subquery is a SELECT statement that is embedded in a clause of another SELECT statement.

They can be very useful when you need to select rows from a table with a condition that depends on the data in the table itself.

You can place the subquery in a number of SQL clauses:

WHERE clause

HAVING clause

FROM clause

Using a Subquery

```
SQL> SELECT ename
  2  FROM   emp
  3  WHERE  sal >
  4                (SELECT sal
  5           FROM   emp
  6           WHERE  empno=7566);
```

2975

In the example, the inner query determines the salary of employee 7566. The outer query takes the result of the inner query and uses this result to display all the employees who earn more than this amount.

```
ENAME
----------
KING
FORD
SCOTT
```

Guidelines for Using Subqueries

- Enclose subqueries in parentheses.
- Place subqueries on the right side of the comparison operator.
- Do not add an ORDER BY clause to a subquery.
  - Subquery Results Are Unordered by Default
  - adding an ORDER BY clause to a subquery is either redundant or ignored, leading to unnecessary complexity and potential performance issues

- Use single-row operators with single-row subqueries.
- Use multiple-row operators with multiple- row subqueries.

Types of Subqueries

- Single-row subquery : Queries that return only one row from the inner SELECT statement



- **Multiple-row subquery** Queries that return more than one row from the inner SELECT statement



- **Multiple-column subquery** Queries that return more than one column from the inner SELECT statement



Single-Row Subqueries

- Return only one row
- Use single-row comparison operators

Operator	Meaning
=	Equal to
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
<>	Not equal to

Display the employees whose job title is the same as that of employee 7369.

```
SQL> SELECT    ename, job
  2  FROM      emp
  3  WHERE     job =
  4                 (SELECT  job
  5                       FROM      emp
  6                       WHERE     empno = 7369);
```

ENAME	JOB
JAMES	CLERK
SMITH	CLERK
ADAMS	CLERK
MILLER	CLERK

Executing Single-Row Subqueries

The example on the slide displays employees whose job title is the same as that of employee 7369 and whose salary is greater than that of employee 7876.

```
SQL> SELECT    ename, job
2   FROM      emp
3WHERE        job =
4                    (SELECT      job
5                    FROM         emp
6                    WHERE        empno = 7369)
7   AND       sal >
8                    (SELECT      sal
9                    FROM         emp
10                   WHERE        empno = 7876);
```

CLERK

1100

```
ENAME      JOB
---------- ---------
MILLER     CLERK
```

Using Group Functions in a Subquery

Display the employee name, job title, and salary of all employees whose salary is equal to the minimum salary.

```
SQL> SELECT ename, job, sal
  2  FROM   emp
  3  WHERE  sal =
  4                (SELECT   MIN(sal)
  5                FROM                emp);
```

800

```
ENAME      JOB              SAL
---------- --------- ----------
SMITH      CLERK            800
```

HAVING Clause with Subqueries

- The Oracle Server executes subqueries first.
- The Oracle Server returns results into the HAVING clause of the main query.

The SQL statement on the slide displays all the departments that have a minimum salary greater than that of department 20.

```
SQL> SELECT        deptno, MIN(sal)
  2  FROM          emp
  3  GROUP BY      deptno
  4  HAVING        MIN(sal) >
  5
  6
  7
```

```
(SELECT     MIN(sal)
FROM        emp
WHERE       deptno = 20);
```

What Is Wrong with This Statement?

```
SQL> SELECT empno, ename
  2  FROM   emp
  3  WHERE  sal =
  4                (SELECT   MIN(sal)
  5                FROM      emp
  6                GROUP BY  deptno);
```

Single-row operator with multiple-row subquery

```
ERROR:
ORA-01427: single-row subquery returns more than
one row

no rows selected
```

Errors with Subqueries

One common error with subqueries is more than one row returned for a single-row subquery.

In the SQL statement on the slide, the subquery contains a GROUP BY (deptno) clause, which implies that the subquery will return multiple rows, one for each group it finds. In this case, the result of the subquery will be 800, 1300, and 950.

The outer query takes the results of the subquery (800, 950, 1300) and uses these results in its WHERE clause. The WHERE clause contains an equal (=) operator, a single-row comparison operator expecting only one value. The = operator cannot accept more than one value from the subquery and hence generates the error.

To correct this error, change the = operator to IN.

Will This Statement Work?

```
SQL> SELECT ename, job
  2  FROM   emp
  3  WHERE  job =
  4                (SELECT    job
  5                FROM       emp
  6                WHERE      ename='SMYTHE');
```

```
no rows selected
```

Subquery returns no values

Problems with Subqueries

A common problem with subqueries is no rows being returned by the inner query.

In the SQL statement on the slide, the subquery contains a WHERE (ename='SMYTHE') clause. Presumably, the intention is to find the employee whose name is Smythe. The statement seems to be correct but selects no rows when executed.

The problem is that Smythe is misspelled. There is no employee named Smythe. So the subquery returns no rows. The outer query takes the results of the subquery (null) and uses these results in its WHERE clause. The outer query finds no employee with a job title equal to null and so returns no rows.

Multiple-Row Subqueries

- Return more than one row
- Use multiple-row comparison operators

Operator	Meaning
IN	Equal to any member in the list
ANY	Compare value to each value returned by the subquery
ALL	Compare value to every value returned by the subquery

Using ANY Operator in Multiple-Row Subqueries

Display the employees whose salary is less than any clerk and who are not clerks.

```
SQL> SELECT   empno, ename, job
  2  FROM     emp
  3WHERE      sal < ANY
  4                       (SELECT     sal
  5                       FROM emp
  6                       WHERE       job = 'CLERK')
  7  AND        job <> 'CLERK';
```

1300
1100
800
950

```
     EMPNO ENAME      JOB
---------- ---------- ---------
      7654 MARTIN     SALESMAN
      7521 WARD       SALESMAN
```

<ANY means less than the maximum. >ANY means more than the minimum. =ANY is equivalent to IN.

Using ALL Operator in Multiple-Row Subqueries

Display the employees whose salary is greater than the average salaries of all the departments.

```
SQL> SELECT   empno, ename, job
  2  FROM     emp
  3  WHERE    sal > ALL
  4                  (SELECT   avg(sal)
  5                  FROM                 emp
  6                  GROUP BY deptno);
```

1566.6667
2175
2916.6667

```
     EMPNO ENAME      JOB
---------- ---------- ---------
      7839 KING       PRESIDENT
      7566 JONES      MANAGER
      7902 FORD       ANALYST
      7788 SCOTT      ANALYST
```

>ALL means more than the maximum and <ALL means less than the minimum.The NOT operator can be used with IN, ANY, and ALL operators.

Summary

- Subqueries are useful when a query is based on unknown values.

```
SELECT      select_list
FROM        table
WHERE       expr operator
                    (SELECT select_list
                     FROM   table);
```

Reasons why subqueries are useful for unknown values:

1.Dynamic Value Retrieval: Subqueries allow the main query to use data that is computed or retrieved on-the-fly by another query. For example, if you need to select all employees whose salaries are higher than the average salary, you can use a subquery to first calculate the average salary.

2. **Nested Logic:** Subqueries can be used in the `WHERE`, `HAVING`, or `FROM` clauses to enable complex filtering and data selection logic without knowing the values beforehand.

3. **Modularity:** Subqueries can break down complex problems into smaller, modular parts, making the query easier to write and understand. They allow you to focus on retrieving necessary information step-by-step without explicitly knowing the final values.

4. **Handling Relationships:** Subqueries allow you to handle relationships between tables dynamically, even if you don’t know the exact relationships ahead of time. For example, if you need to select customers who have made the most purchases, a subquery can dynamically find the top customer and then use that information in the main query.

5. **Conditional Logic:** Subqueries can adapt based on conditional logic, enabling the query to fetch appropriate results based on various conditions. For instance, fetching products priced higher than the average for a specific category.